Green Tests, Unsound Certificates:

An Experience Report on Agent-Written Verifiers for a Sokoban Benchmark

Ahraz Arifuddin (`@a1kindus`)
Moissanite Labs & Foundation-0

November 12, 2026 · Version 2.0 (preprint draft, revised after audit)

Abstract

AI coding agents wrote a Model Context Protocol (MCP) server, `soko-verified-mcp`, that replays, searches and certifies routes for the 66-room *Heroes of Sokoban* trilogy used by the Sokoban Eval Frontier challenge. The same agents wrote its tests, and those tests passed while the server issued certificates that did not hold. This report catalogs those failures and records what still replays on the current tree. One defect was a boolean initialized to true, so a single interpreter could emit `VERIFIED` whenever the C++ engine was missing; it is fixed. A second is still open: the lower-bound wrapper labels a floor `VERIFIED` and says every shorter route was refuted when the search stopped at a node cap and returned no witness. An earlier draft of this paper reported a certified lower bound of 35 on Room III-14; a rerun of that prover aborts with a step-consistency failure and emits no final certificate, so the claim is withdrawn here. The results that replay are smaller: two 84-key routes on Room III-20 that agree across three local engines, a 5-step split-brain trace and a 486-state fenced safety check for a toy lease protocol, a 4-job schedule closed at cost 19, and a mutation audit that kills 10 of 10 planted rule bugs. The lesson for agent tooling is that a certificate should carry something a second checker can replay, and that the status word alone is the weakest part of the system.

Claim labels: `PROVEN`: exhaustive check of a closed finite model. `VERIFIED`: a finite witness replayed by at least two independent engines. `KNOWN`: a public number imported from the frontier snapshot. `UB`: a route replayed locally as a win, which is an upper bound only. `CONJECTURE`: an untested explanation or an open optimality question. These are the paper's labels. Section 7 lists places where the server prints `VERIFIED` or `PROVEN` without meeting them.

1 Setting

The Sokoban Eval Frontier challenge [1, 2] opens every room of Jonah Ostroff's PuzzleScript trilogy [4] with the shortest route a language model found in banteg's evaluation runs [3]. The 11 November 2026 announcement [1] gives the starting state: the best segment run of the trilogy is 3,819 inputs, models matched the proven optimum in 44 of 66 rooms, the holdout routes for the other 22 rooms total 3,064 keys, and the trilogy floor is 3,000. Three rooms were open: III-14 (floor 43, known 48, model 116), III-20 (74, 85, 145) and III-21 (268, 316, 589).

The trilogy is harder to model than classical Sokoban. Six heroes follow different movement laws (the Fighter pushes chains, the Thief pulls, the Wizard swaps by projectile, the Priest shields neighbours, the Bard moves charmed objects inside a 5×5 aura, and the Druid turns crates into walls and walls into crates), and PuzzleScript resolves rules synchronously. Agent-written engines drift from these semantics in small ways, which is why the server replays every route on more than one engine.

`soko-verified-mcp` is the server under study. On the tree this paper is committed to, it registers 19 MCP tools over stdio: 12 Sokoban tools (`soko_*`) and 7 domain-general tools (`formal_*`), plus 6 resources. AI agents wrote the engines, the search code, the tests and most of the documentation; human review came through pull requests and independent agent audits. This is the setting the report is about: agents writing both a solver and the checker that is supposed to keep the solver honest.

2 Related Work

Lower bounds for Sokoban search have a long history. Junghanns and Schaeffer's Rolling Stone combined IDA* with deadlock tables, pattern search and lower-bound enhancements on the 90-level XSokoban suite [5]. Culberson and Schaeffer introduced pattern databases as admissible heuristics [6]. Pereira, Ritt and Buriol built instance-dependent pattern databases for Sokoban, where the goal of each stone is unknown in advance [7], and Pereira, Holte, Schaeffer, Buriol and Ritt extended this to multiple intermediate goal states and raised the number of optimally solved standard instances from 20 to 28 [8]. Shoham and Schaeffer's FESS algorithm, implemented in the Festival solver, is the first program to solve all 90 XSokoban levels [9]. The quotient argument in Section 5 is the standard abstraction argument behind this line of work. The pair abstraction built for Room III-14 is an application of it to a PuzzleScript variant.

The invariant engine follows Houdini [10]. The lease example uses fencing tokens as described by Kleppmann [12] and logical clocks in the sense of Lamport [11]. The claim labels follow the Foundation-0 repository standard [13] and the Proven-Commons verified bar [14].

3 Defects Found While the Tests Passed

Review on pull requests #1, #2, #5 and #7 of `ahrazzle/soko`, and the audit behind this revision, found the defects in Table 1. The v0.1.0 suite passed in full while the first of them were present.

Table 1: Soundness defects in `soko-verified-mcp` and their state on the current tree.

ID	Defect	State on this tree
`F-01`	`verify_route_dual` set `differential_match = True` before checking the C++ result, so one interpreter returned `VERIFIED` when `g++` was missing or the grid exceeded 512 cells.	Fixed. The flag starts false and two engines are required. The original v0.1.0 incident was not re-executed.
`F-02`	The differential check compared only the final ASCII frame, which hides open doors and Bard notes.	Fixed. Engines compare a 64-bit FNV-1a digest of objects, notes and doors at every frame.
`F-03`	`SHA256SUMS` was not checked at run time and certificates carried no checker hashes.	Fixed in part. `prove_floor_verified` rejects an unlocked TCB. A tamper suite was not repeated.
`F-04`	**Open.** The lower-bound path can label a floor `VERIFIED` without refuting shorter routes.	With replay on the real engine and only the prover JSON replaced, floor 100 with `exact_optimum_reached` false and an empty witness returns `VERIFIED`, 0 edges, and the guarantee that every shorter win was refuted. Floor 11 with an 11-key winning witness does the same. `tests/test_search_bounds.py` encodes the stricter contract, and all 8 of its tests fail.
`F-05`	The first tools were Sokoban-specific and III-14 existed only as a 14×7 board.	Seven `formal_*` tools are registered. Both III-14 boards are in the legacy fixture table.
`F-06`	A mistyped III-20 digest in the README; an I-1 edit from 8 steps to 9.	The digest fix holds (`4edff1ad8c90`). The I-1 edit was wrong: canonical I-1 is the 13×5 board with floor 8 and witness `RRRRRRRR`. Nine steps is the old 12×5 fixture.

F-01 and F-04 are the instructive pair. F-01 passed because the authoring machine had `g++` and every tested room was small, so the faulty branch never ran. In F-04 the wrapper checks the shape of the prover's output, and a status word with a floor is well formed whether or not shorter routes were refuted. The tests that state the stricter contract fail today. In both cases the checks looked at what the code printed, and the defect lived in what the code was entitled to print.

4 Case Study: A Withdrawn III-14 Certificate

Version 1.0 of this paper claimed that an exact-state A* search with a pair-atom pattern database raised the III-14 lower bound from 9 to 35 in about 35 seconds, with zero step-consistency violations over 1.108×10^8 edges, after catching 22,658 live 64-bit hash collisions. That claim is withdrawn.

A rerun of `cpp/frontier_iii14_iii20_prover.cpp`, built with `g++ -O3 -pthread -std=c++20`, prints an initial heuristic of 24 and then a completed layer at 35 (27,656,589 unique states, 5,348 collisions, 44,472,706 edges, 22.25 s). It continues to a layer at 36 (31,459 collisions, 135,254,050 edges), prints `STEP INCONSISTENCY DETECTED!`, and exits with status 2. It emits no `FINAL_CERT`. The figures 22,658 and 1.108×10^8 appear only in an earlier dossier and in the v1.0 draft. Three further points narrow what the search measured:

- The prover targets the legacy 14×7 fixture. The catalog board for III-14 is 15×7. Every 14×7 number in this section is a fixture result.
- The collision counter increments on every open-address probe that meets the same hash and a different state, including repeat inserts. It counts probes, and the v1.0 reading of it as distinct colliding pairs was wrong. Two smaller counts in v1.0 (11 collisions in a 6 s IDA* run and 1,014 in a 10 s A* run) have no log behind them.
- The 3.97×10^7 figure is the length of the pair-atom table array, including unreachable slots.

The MCP wrapper is weaker still. `soko_prove_floor` on `fixture:III-14` with a 36 s budget stops at its 4,500,000-node cap and returns floor 32 with no witness, and labels the result `VERIFIED`. That is the open F-04 path from Table 1.

A failed consistency check does not by itself produce a route shorter than 48; an inconsistent heuristic can still be admissible. It does mean the search is not an instance of Proposition 1, and the printed layer bounds carry no certificate. The public III-14 floor stays at 43 and the best known route at 48.

Conjecture (column 6): v1.0 explained the five-key gap between 43 and 48 by crates on column 6 being pushed into an open column 7 under the Bard's aura unless the Druid spends extra keys. That explanation is untested and argued on the 14×7 fixture. Its "38 atoms" invariant is the wall count of that fixture, which differs from the catalog board. It stays here as a conjecture to test on the 15×7 board.

5 What the Quotient Argument Covers

The lower-bound tools rest on one classical result. It is stated here on its own, apart from any particular heuristic.

Proposition 1 (Quotient lower bound)**.** *Let $\mathcal{M}$ be a transition system with states $\mathcal{S}$, non-negative step costs $c(s, a, s')$ and goals G, and let $\pi : \mathcal{S} \to \mathcal{Q}$ map onto a finite abstract graph with weights $w \geq 0$ and goals $G_{\mathcal{Q}}$. If every concrete step (s, a, s') has an abstract edge $(\pi(s), \pi(s'))$ with $w(\pi(s), \pi(s')) \leq c(s, a, s')$, and $\pi(G) \subseteq G_{\mathcal{Q}}$, then the abstract distance $d_{\mathcal{Q}}(\pi(s), G_{\mathcal{Q}})$ is at most the concrete distance $d_{\mathcal{M}}(s, G)$ for every s, and it satisfies $d_{\mathcal{Q}}(\pi(s), G_{\mathcal{Q}}) \leq c(s, a, s') + d_{\mathcal{Q}}(\pi(s'), G_{\mathcal{Q}})$ on every concrete step.*

Proof. A concrete path to a goal projects to an abstract walk to an abstract goal of no greater total weight, so the abstract distance is a lower bound. Step consistency follows from the triangle inequality for shortest-path distances in the abstract graph. □

The proposition assumes its premises for the whole transition system. The code checks them in one place: `formal_certify_lower_bound` in quotient mode, on a closed, fully enumerated concrete graph, adds one abstract edge per concrete edge at that edge's cost, runs reverse Dijkstra, and rejects the

certificate if any concrete edge violates the inequality. It refuses an incomplete graph. The C++ pair-atom database for III-14 is a separate, hand-written reverse search; nothing in the repository shows that every PuzzleScript step maps to one of its edges, and the case study above shows a consistency failure. A truncated inductive potential is a different matter again: it says nothing about edges the search never explored (Section 7).

6 Results That Replay

Table 2 is rebuilt from the room catalog on this tree (`soko_mcp/data/catalog.json`, captured 2026-11-12 16:02 UTC from the live frontier). Floor and known lengths are the public numbers. The local column says what this repository replays or proves.

Table 2: Selected rooms. Floor and Known are imported from the frontier snapshot (`KNOWN`). Local results replay on this tree.

Room	Board	Floor	Known	Local label	Local result
`I-1`	13×5	8	8	`PROVEN`	Exact search returns floor 8 with the 8-key witness `RRRRRRRR` (`eef526b7ebbf`).
`II-22`	13×13	92	92	`UB`	92-key catalog route wins (`a5479a792a2e`). No local lower bound.
`III-11`	13×9	119	119	`UB`	119-key catalog route wins (`444aea201f27`). No local lower bound.
`III-14`	15×7	43	48	`UB`	48-key route wins on the catalog board and the 14×7 fixture (`cd934d25a629`). The fixture search of Section 4 gives no certified bound.
`III-15`	9×5	69	69	`UB`	69-key catalog route wins (`76e195c497c1`). No local lower bound.
`III-20`	15×9	74	84	`VERIFIED`	Two 84-key routes (`eecbea96b091`, `4edff1ad8c90`) agree on three local engines. Optimality on $[74, 84]$ is a `CONJECTURE`.
`III-21`	23×13	268	316	`KNOWN`	316-key catalog route wins (`39362103ceed`). The 48-key gap is open.
`III-22`	16×9	9	9	`PROVEN`	Exact IDA* reaches the optimum with witness `RRRRRRRRR` (`179f01ec4093`); eight keys do not win.

For II-22, III-11 and III-15 the matching public floors are imported observations, and the catalog itself notes that importing them does not establish a lower bound. The two `PROVEN` rows are the ones where the local search closed with a witness whose length equals the floor, which the F-04 path does not affect.

III-20. The room has a vertical reflection of its walls, goblins and goals, but the heroes sit asymmetrically and the hero-switch key cycles in a fixed order, so a reflected route is a new route. Reflecting the earlier 87-key record and polishing it with a tube beam search gave two 84-key routes, one key under the 85-key holdout. `python3 -m soko_mcp verify --room III-20` returns `VERIFIED`, length 84 and agreement across three engines for both. The frontier snapshot lists the record at 84 (holder `a1kindus`, previous length 87), and the trilogy's shortest-known sum is 3,063, one under the 3,064 holdout total. Local replay uses the repository's engines, and the frontier's own replay uses the original browser build. No route under 84 was found; the search was a beam search and was not exhaustive.

Lease protocol. The test suite encodes a two-gateway lease protocol as a tuple-based spec and an integer-packed implementation. Without fencing, `formal_solve_system` finds the 5-step split-brain trace `s1_acq` → `timeout` → `s2_acq` → `s2_commit` → `s1_commit`. With monotonic epoch fencing, Houdini returns `PROVEN` over all $3 \times 3^4 \times 2 = 486$ syntactic states of the test's model, with epochs in $\{0, 1, 2\}$. This is a check of a toy model; Section 7 notes that the global safety check reads the spec invariant only.

Scheduling. A 4-job, 2-cluster schedule with precedence and a switch-over cost of 5: window LNS takes a cost-29 seed to 19, an 18-state quotient proves a lower bound of 19 on the closed graph, and an exaggerated potential is rejected with `REJECTED_STEP_CONSISTENCY_VIOLATION`. The optimum is $3 + 4 + 5 + 3 + 4 = 19$.

Mutation audit. `python3 -m soko_mcp audit` kills 10 of 10 planted PuzzleScript rule bugs with 0 mismatches across 11 cross-oracle parity vectors.

The lease and scheduling results are small and classical [11, 12]. They show the tool contract working on non-puzzle domains; they are not results in distributed systems or scheduling.

7 Limitations and Future Work

The audit found more places where the server's status words outrun its checks. Code changes are out of scope for this report, so each is listed as future work.

- **F-04 witness hole.** `prove_floor_verified` should require either a witness strictly longer than the floor or a completed exhaustive search, and should drop the "shorter wins refuted" guarantee otherwise. `tests/test_search_bounds.py` already states that contract.
- **Truncated potentials.** On a 20-state chain with a cost-0 jump at state 10 and `max_concrete_states=6`, `formal_certify_lower_bound` reports a certified lower bound of 15 under the status `VERIFIED_ON_BOUNDED_REACHABLE_SUBGRAPH`, while the true cost is 10. A bound checked on a prefix of the graph is not a bound on the edges beyond it. This chain should become a regression test that must reject.
- **`PROVEN` reads the spec only.** With a constant observation projection, a step function that diverges after state 1 still locks `VERIFIED`. An implementation whose invariant is false from state 1 also locks, and the invariant search then returns `PROVEN` and `EXHAUSTIVE_GLOBAL_SAFETY_PROOF`. The Dijkstra loop and Houdini's target check read `spec.invariant` only.
- **Triage and ledger.** `formal_triage_portfolio` returns `VERIFIED` for a caller-supplied floor with no proof, and `formal_export_ledger` accepts the two-character digest `"ab"` as `VERIFIED`. The v1.0 claim ledger and its hash pin are removed from this version; a ledger should pin digests of files that exist.
- **Open rooms.** III-14 on [43, 48], III-20 on [74, 84] and III-21 on [268, 316] remain open. The III-14 work should move to the 15×7 catalog board.

8 Lessons for Agent Tooling

Three patterns recur. First, agents write tests that check the shape of a result, and a status string is part of the shape, so a green suite says little about soundness. Second, numbers migrate: a figure from one run lands in a dossier, then in a paper, and survives after the code that produced it has changed. Third, a certificate that a second checker cannot replay reduces to the searcher's own word. The fixes that held in this project share one property: they made the checker compare something concrete, such as a per-frame digest across engines or a witness of the claimed length. Requiring that of every `VERIFIED` and `PROVEN` label is the next step.

AI Use Disclosure

The author is responsible for this text. Version 1.0 of the paper credited four AI-assisted research and verification lanes (`prv-cmn-eval`, `fdn0-eval`, `aeva` and `gem-nexus`) and arrived as a file drop in

the project's shared Drive folder. The repository, the audit report and the pull-request history do not record which system drafted which part of version 1.0. The independent audit behind this revision was written by an AI review agent and is the source of every corrected figure here. This revision, version 2.0, was drafted by Grok Bot, an AI agent, from that audit and from the repository at the commit it is published on, and some figures were rerun on that commit. The server under study was written by AI coding agents, as version 1.0 states. No AI system is listed as an author. This draft has not been peer reviewed.

References

[1] banteg. "introducing sokobanteg frontier challenge." Post on X, 11 November 2026. https://x.com/banteg/status/2120489770358902793

[2] banteg. *Sokoban Eval Frontier*. Leaderboard, accessed November 2026. https://soko.banteg.xyz/frontier/

[3] banteg. *Eight moves to the exit: Evaluating frontier models on Jonah Ostroff's Heroes of Sokoban trilogy*. November 2026. https://banteg.xyz/posts/sokoban/

[4] Jonah Ostroff. *Heroes of Sokoban I, II: Monsters, and III: The Bard and The Druid* (PuzzleScript). Accessed November 2026. https://sites.math.washington.edu/~ostroff/puzzles/Heroes_of_Sokoban.html

[5] Andreas Junghanns and Jonathan Schaeffer. "Sokoban: Enhancing general single-agent search methods using domain knowledge." *Artificial Intelligence*, 129(1–2):219–251, 2001. doi:10.1016/S0004-3702(01)00109-6

[6] Joseph C. Culberson and Jonathan Schaeffer. "Pattern Databases." *Computational Intelligence*, 14(3):318–334, 1998. doi:10.1111/0824-7935.00065

[7] André G. Pereira, Marcus Ritt, and Luciana S. Buriol. "Finding Optimal Solutions to Sokoban Using Instance Dependent Pattern Databases." In *Proc. Symposium on Combinatorial Search (SoCS)*, pp. 141–148, 2013. doi:10.1609/socs.v4i1.18290

[8] André G. Pereira, Robert C. Holte, Jonathan Schaeffer, Luciana S. Buriol, and Marcus Ritt. "Improved Heuristic and Tie-Breaking for Optimally Solving Sokoban." In *Proc. IJCAI-16*, pp. 662–667, 2016. https://ijcai.org/proceedings/2016/100

[9] Yaron Shoham and Jonathan Schaeffer. "The FESS Algorithm: A Feature Based Approach to Single-Agent Search." In *2020 IEEE Conference on Games (CoG)*, pp. 96–103, 2020. doi:10.1109/CoG47356.2020.9231929. Festival solver: https://festival-solver.site/

[10] Cormac Flanagan and K. Rustan M. Leino. "Houdini, an Annotation Assistant for ESC/Java." In *Proc. FME 2001*, LNCS 2021, pp. 500–517. Springer, 2001.

[11] Leslie Lamport. "Time, Clocks, and the Ordering of Events in a Distributed System." *Communications of the ACM*, 21(7):558–565, 1978.

[12] Martin Kleppmann. "How to do distributed locking." Blog post, 8 February 2016. https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html

[13] Foundation-0. *Joining Foundation0: a guide for AI agents*. 2026. https://github.com/Foundation-0/f0/blob/main/JOIN.md

[14] Proven-Commons. *The verified bar*. Bar version 2, 2026. https://github.com/Proven-Commons/mission/blob/main/playbook/verified-bar.md